Practical 1: Basic Feature Engineering

OBJECTIVE

To clean a dataset by filling missing values, changing text categories into numbers, and scaling numerical values.

THEORY

Feature engineering means preparing data before giving it to a machine-learning model. Sometimes a dataset has empty values. We can fill these values using the average, middle value, or most common value. Categorical data means values such as gender, city, or ticket class. A machine cannot directly understand text, so we convert it into numbers using encoding. Scaling changes numerical values to a similar range. This helps the model learn better.

ALGORITHM

1. Load a dataset such as Titanic using Pandas.

2. Display the first five rows and check the columns.

3. Check how many missing values are present.

4. Fill missing numerical values using the median.

5. Fill missing categorical values using the most common value.

6. Convert categorical columns into numbers using encoding.

7. Scale numerical columns using StandardScaler.

8. Display the cleaned dataset and check for missing values again.

CONCLUSION

Missing values are filled, categorical data is converted into numbers, and numerical data is scaled. The dataset is now clean and ready for machine-learning work.

Practical 2: Dimensionality Reduction Using PCA and LDA

OBJECTIVE

To reduce the number of features using PCA and LDA and compare the model results before and after reduction.

THEORY

Sometimes a dataset contains many features. Using all features can take more time and may include unnecessary information. Dimensionality reduction means reducing the number of features while keeping useful information. PCA creates new features based on the directions with the highest data variation. It does not use class labels. LDA uses class labels and tries to keep different classes separate. We can train a classifier before and after reduction and compare accuracy and time.

ALGORITHM

1. Load a labelled dataset such as the Wine dataset.

2. Separate input features X and target y.

3. Split the data into training and testing parts.

4. Scale the input features using StandardScaler.

5. Train Logistic Regression using all original features.

6. Apply PCA to reduce the number of features.

7. Train Logistic Regression using the PCA output.

8. Apply LDA and train Logistic Regression using the LDA output.

9. Compare accuracy and training time for all three models.

CONCLUSION

PCA and LDA reduce the number of features. PCA keeps important data variation, while LDA tries to separate classes. Their results can be compared using accuracy and training time.

Practical 3: Multiple Linear Regression

OBJECTIVE

To build a Multiple Linear Regression model and check its prediction results using MSE, RMSE, and R2 score.

THEORY

Multiple Linear Regression is used to predict a number using two or more input features. For example, house price can be predicted using area, number of rooms, and location-related features. The model learns how each input feature affects the output. MSE measures the average squared error between actual and predicted values. RMSE is the square root of MSE and is in the same unit as the target. R2 tells us how well the model explains the target values. A higher R2 is usually better.

ALGORITHM

1. Load a real-world dataset, such as a housing dataset.

2. Select input features and the target column.

3. Check and handle missing values if present.

4. Split the dataset into training and testing parts.

5. Create a LinearRegression model.

6. Train the model using the training data.

7. Predict values for the testing data.

8. Calculate MSE, RMSE, and R2 score.

9. Display and compare the evaluation results.

CONCLUSION

The Multiple Linear Regression model predicts a continuous value using multiple input features. MSE, RMSE, and R2 help us understand how well the model predicts.

Practical 4: Polynomial Regression

OBJECTIVE

To use different polynomial degrees and observe how model complexity affects prediction accuracy.

THEORY

Polynomial Regression is used when the relationship between input and output is not a straight line. It adds powers of the input, such as x2 and x3, to help the model learn a curved pattern. A small degree may be too simple and may not learn the data properly. This is called underfitting. A very large degree may learn even the noise in the training data. This is called overfitting. We compare training and testing errors for different degrees to choose a suitable one.

ALGORITHM

1. Load a suitable open-access regression dataset.

2. Select input feature X and target y.

3. Split the data into training and testing parts.

4. Choose polynomial degrees such as 1, 2, 3, 4, and 5.

5. Convert the input data into polynomial features for each degree.

6. Train a Linear Regression model for each degree.

7. Predict values for training and testing data.

8. Calculate the error for each model.

9. Compare the errors and observe underfitting or overfitting.

CONCLUSION

Polynomial Regression can learn curved relationships. A very low or very high degree may give poor results. Comparing errors helps us choose a suitable degree.

Practical 5: Ridge and Lasso Regression

OBJECTIVE

To apply Ridge and Lasso regression and observe how they help control overfitting and large model coefficients.

THEORY

Ridge and Lasso are methods used to control a regression model. Sometimes a model learns the training data too closely. This is called overfitting. Regularization adds a penalty to large coefficients and helps control this problem. Ridge uses an L2 penalty and makes coefficients smaller. Lasso uses an L1 penalty and can make some coefficients zero. This means Lasso can also help select useful features. Feature scaling is important before applying these methods.

ALGORITHM

1. Load a multivariate regression dataset.

2. Separate input features X and target y.

3. Split the data into training and testing parts.

4. Scale the input features.

5. Train a simple Linear Regression model as a baseline.

6. Train a Ridge Regression model.

7. Train a Lasso Regression model.

8. Predict values using all models.

9. Compare MSE, RMSE, and R2 scores.

10. Check the model coefficients and compare the results.

CONCLUSION

Ridge and Lasso help reduce overfitting by controlling large coefficients. Ridge makes coefficients smaller, while Lasso can also remove less useful features.

Practical 6: Logistic Regression for Binary Classification

OBJECTIVE

To build a Logistic Regression model for two-class classification and evaluate it using a confusion matrix, precision, recall, and F1-score.

THEORY

Logistic Regression is used to classify data into two classes, such as yes/no or positive/negative. It predicts the probability of a class. A threshold, often 0.5, is used to decide the final class. A confusion matrix shows correct and incorrect predictions. Precision tells us how many predicted positive cases were actually positive. Recall tells us how many actual positive cases were found. F1-score combines precision and recall into one value.

ALGORITHM

1. Load a binary classification dataset, such as Breast Cancer Wisconsin.

2. Separate input features X and target y.

3. Split the data into training and testing parts.

4. Scale the input features.

5. Create and train a Logistic Regression model.

6. Predict class labels for the testing data.

7. Display the confusion matrix.

8. Calculate precision, recall, and F1-score.

9. Study the results and note the correct and incorrect predictions.

CONCLUSION

Logistic Regression is useful for two-class classification. The confusion matrix, precision, recall, and F1-score help us check the model's performance.

Practical 7: Support Vector Machine with Different Kernels

OBJECTIVE

To train SVM models using linear and nonlinear kernels and compare their classification results.

THEORY

Support Vector Machine (SVM) is a classification algorithm. It finds a boundary that separates different classes. The linear kernel is useful when classes can be separated using a straight boundary. The RBF kernel can make curved or more complex boundaries. SVM uses important data points near the boundary to decide the separation. Scaling the features is important because SVM uses distances between data points.

ALGORITHM

1. Load a classification dataset such as the Breast Cancer dataset.

2. Separate input features and target labels.

3. Split the data into training and testing parts.

4. Scale the input features.

5. Create an SVM model using the linear kernel.

6. Create another SVM model using the RBF kernel.

7. Train both models using the training data.

8. Predict the test labels using both models.

9. Compare accuracy, precision, recall, and F1-score.

CONCLUSION

SVM can classify data using different kernels. The linear kernel makes a straight boundary, while the RBF kernel can make a curved boundary. The evaluation scores help us compare them.

Practical 8: K-Means Clustering

OBJECTIVE

To group similar data points using K-Means and find a suitable number of clusters using the Elbow or Silhouette method.

THEORY

K-Means is an unsupervised learning algorithm. It groups similar data points into K clusters. First, it selects K centre points called centroids. Each data point is assigned to its nearest centroid. Then the centroids are updated using the average of the points in each cluster. This process repeats until the clusters stop changing. The Elbow method uses a graph of K and inertia. The bend in the graph can suggest a good number of clusters. The Silhouette score checks how well-separated the clusters are.

ALGORITHM

1. Load a numerical dataset such as the Iris dataset.

2. Select the numerical features.

3. Scale the features.

4. Run K-Means for different K values, such as 1 to 10.

5. Store the inertia value for each K.

6. Plot the Elbow graph.

7. Find the bend in the graph to choose a suitable K.

8. Train the final K-Means model using the selected K.

9. Display the cluster labels and visualize the clusters.

CONCLUSION

K-Means groups similar data points together. The Elbow or Silhouette method helps us select a suitable number of clusters.

Practical 9: DBSCAN Clustering

OBJECTIVE

To use DBSCAN to find groups of nearby data points and identify noise points.

THEORY

DBSCAN is a clustering algorithm that makes clusters based on the density of nearby data points. It does not need the number of clusters in advance. It uses two main values: eps and min_samples. Eps is the distance around a point. Min_samples is the minimum number of nearby points needed to form a dense area. DBSCAN identifies core points, border points, and noise points. Noise points do not belong to any cluster and are usually marked as -1. Scaling is important because the algorithm uses distances.

ALGORITHM

1. Load a numerical dataset such as the UCI Seeds dataset.

2. Select numerical features and handle missing values.

3. Scale the features.

4. Choose values for eps and min_samples.

5. Create and train a DBSCAN model.

6. Get the cluster labels for all data points.

7. Count the clusters, ignoring label -1.

8. Count the noise points labelled as -1.

9. Visualize the clusters and try different parameter values.

CONCLUSION

DBSCAN finds clusters based on data density and marks isolated points as noise. The result depends on eps and min_samples values.

Practical 10: Ensemble Learning

OBJECTIVE

To compare an ensemble model, such as Random Forest or Boosting, with a simple machine-learning model.

THEORY

Ensemble learning combines many models to get better predictions. Random Forest builds many decision trees and combines their results. For classification, it usually selects the class predicted by most trees. Boosting builds models one after another. Each new model tries to improve the mistakes made by earlier models. A simple model, such as a single Decision Tree, can be used as a baseline. We compare both models using the same test data and evaluation measures.

ALGORITHM

1. Load a suitable open-access classification dataset.

2. Separate input features and target labels.

3. Split the data into training and testing parts.

4. Train a simple baseline model, such as Decision Tree.

5. Train an ensemble model, such as Random Forest.

6. Predict the test labels using both models.

7. Calculate accuracy, precision, recall, and F1-score.

8. Compare the results of both models.

9. Write which model performed better on the selected dataset.

CONCLUSION

Ensemble learning combines multiple models and may give better results than a single model. Comparing the scores shows whether the ensemble model improves performance.

Practical 11: Q-Learning for Grid World

OBJECTIVE

To use Q-Learning in a simple Grid World problem and understand exploration, exploitation, and the best path.

THEORY

Reinforcement Learning is a method where an agent learns by taking actions and receiving rewards. In a Grid World, the agent moves from a starting position to a goal. Q-Learning helps the agent learn which action is best in each position. It stores action values in a Q-table. Exploration means trying new actions. Exploitation means choosing the action that currently has the highest value. The epsilon-greedy method balances these two choices. Over many attempts, the agent learns a good path to the goal.

ALGORITHM

1. Create a small Grid World with a start point and a goal point.

2. Decide the possible actions: up, down, left, and right.

3. Give a positive reward for reaching the goal and penalties for unwanted moves.

4. Create a Q-table and fill it with zeros.

5. Set learning rate, discount factor, and exploration rate.

6. Start an episode and choose an action using the epsilon-greedy method.

7. Move the agent and observe the reward and next position.

8. Update the Q-table using the Q-Learning formula.

9. Repeat for many episodes and slowly reduce epsilon.

10. After training, choose the action with the highest Q-value to show the learned path.

CONCLUSION

Q-Learning helps an agent learn a good path by using rewards and repeated practice. The epsilon-greedy method balances trying new actions and choosing the best-known action.